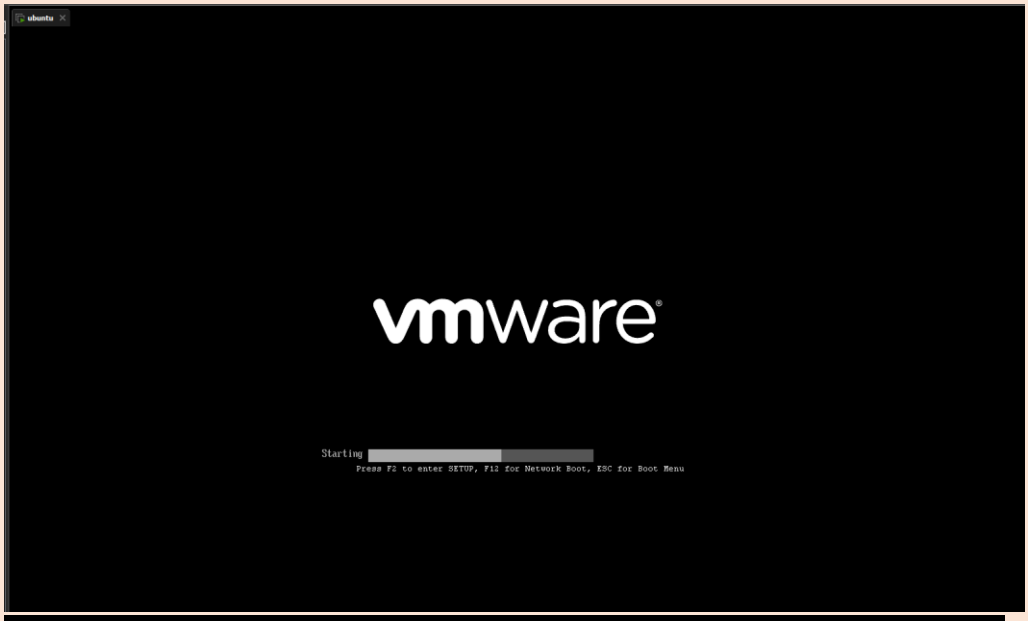
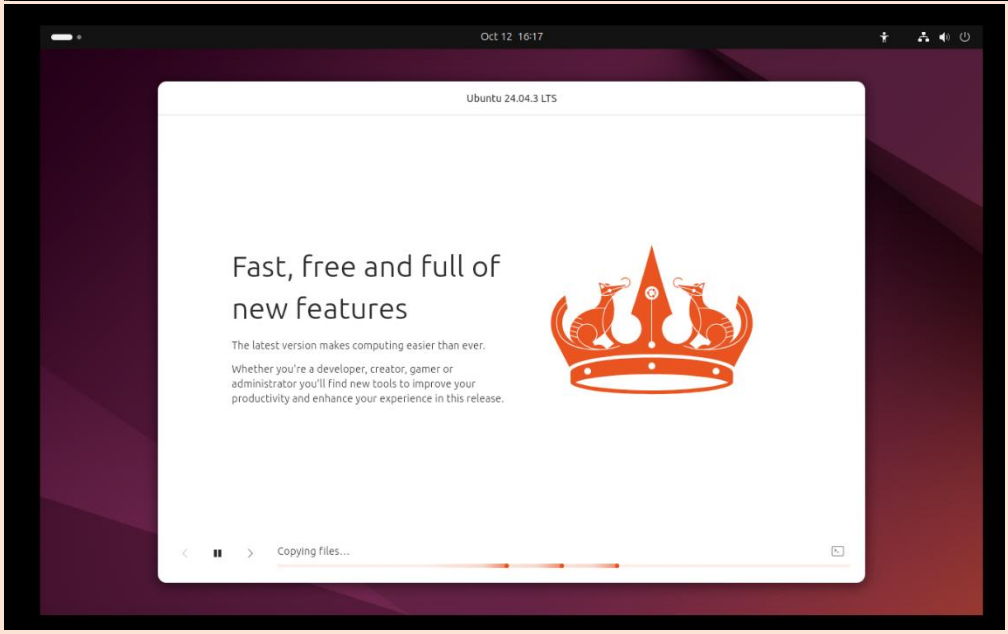
Docker-1

1. Setup a virtual machine using Ubuntu image.
   Objective: To launch an Ubuntu-based virtual machine on vm ware

   Purpose: To create a Linux environment that can be used

   installed vmware software to install ubuntu
   downloaded ubuntu image from the https://ubuntu.com/download
   installed ubuntu image in the vm ware

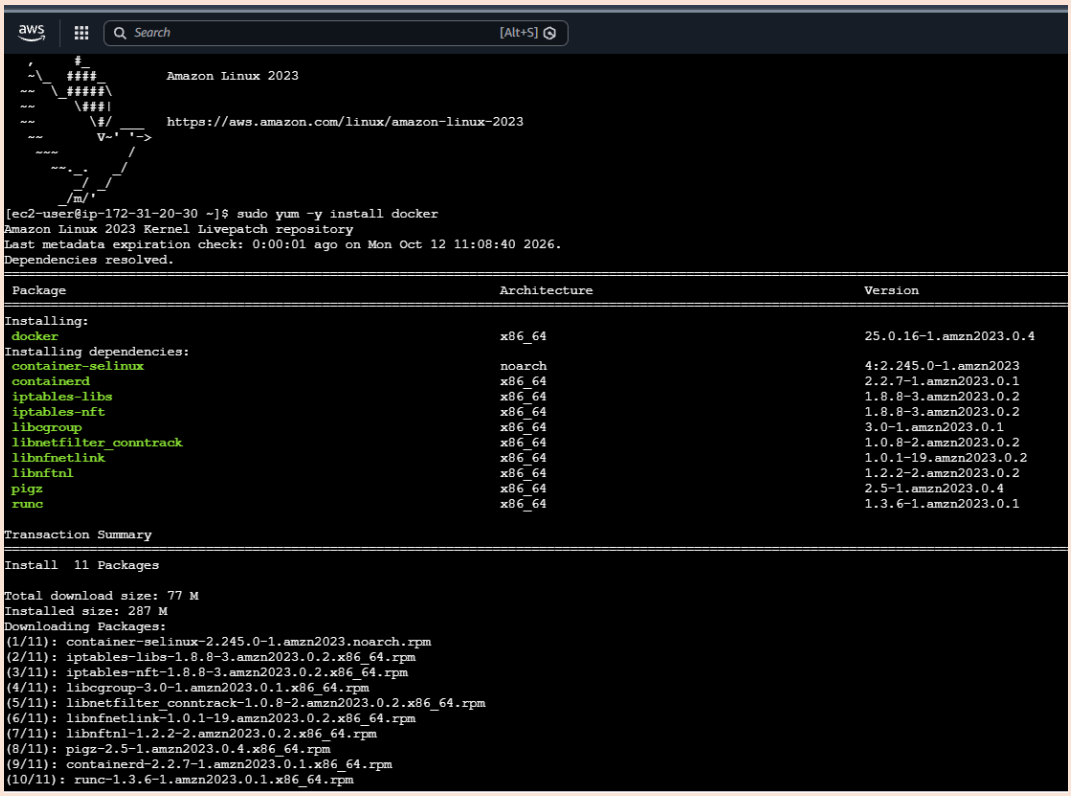


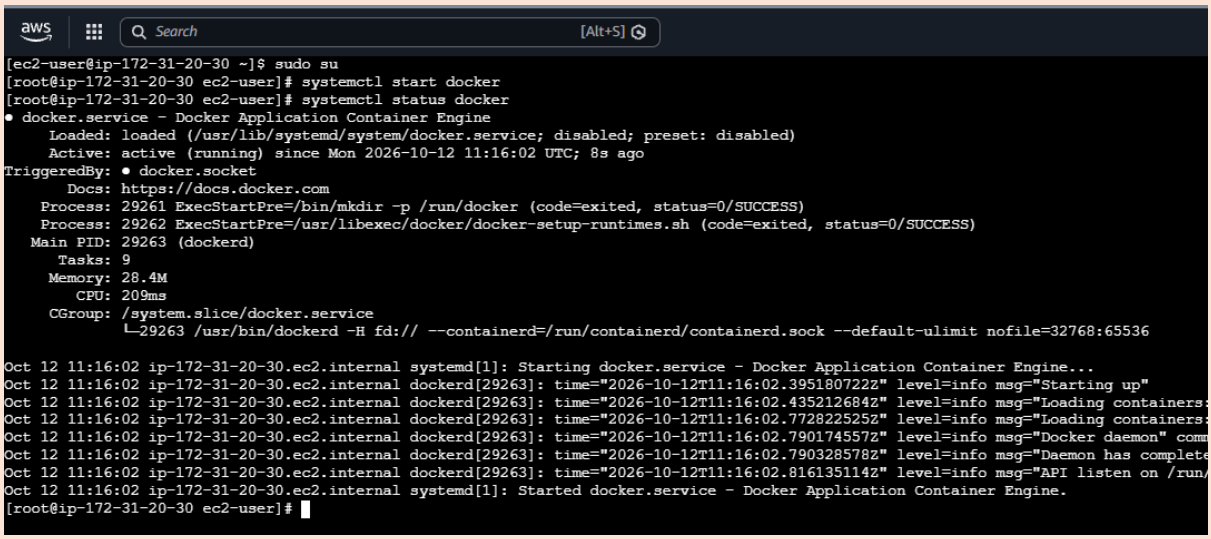


Conclusion: An Ubuntu was created successfully and is ready to be used

2. **Install docker on ec2.**
   Objective: To install and configure Docker on the Ubuntu EC2 instance.
   Purpose: To enable the creation, execution, and management of Docker images and containers on the virtual machine.

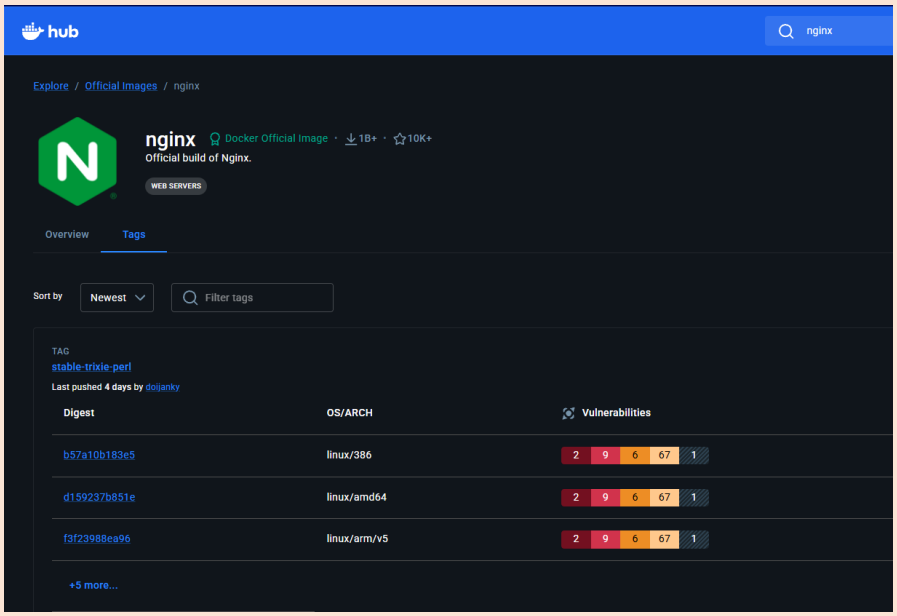
   Go to: Launch an Instance in Ec2
   switch to root user: Sudo su -
   yum -y install docker
   systemctl start docker
   systemctl status docker
   systemctl enable docker

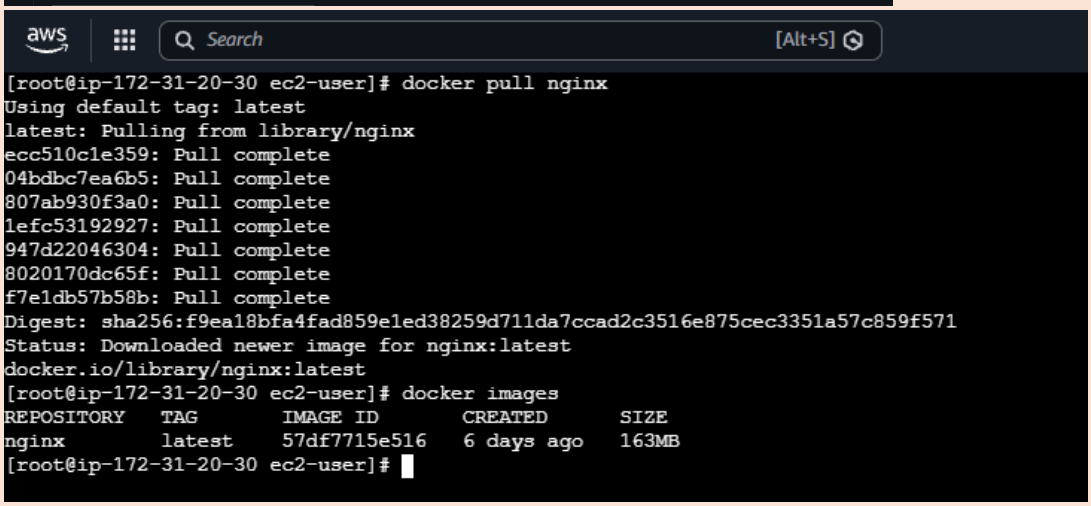




3. Pull 5 docker images. (nginx, Apache tomcat, ubuntu, Jenkins, SonarQube)
   Objective: To download the Nginx, Apache Tomcat, Ubuntu, Jenkins, and SonarQube Docker images from a container registry.

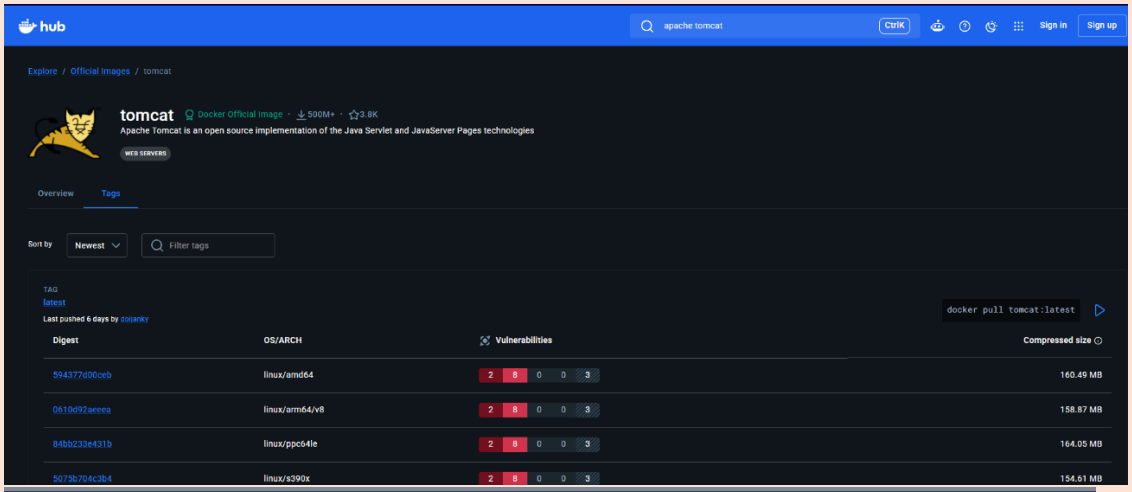
   Purpose: To obtain the required images for practicing web server deployment, application server usage, Linux environments, CI/CD automation, and code quality analysis.
   ssh to ec2 then use this commands
   → docker pull nginx
   → docker images

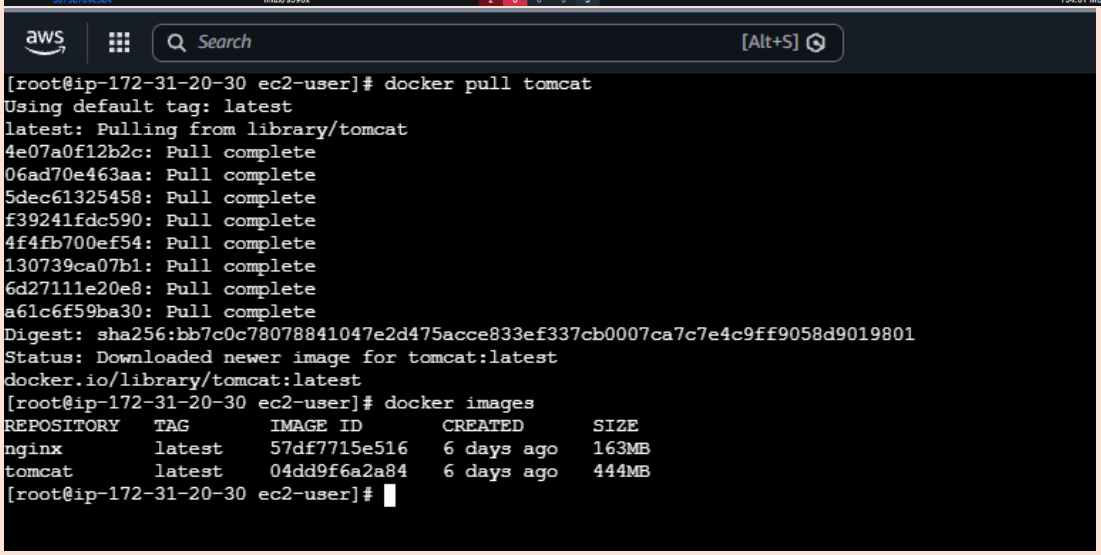




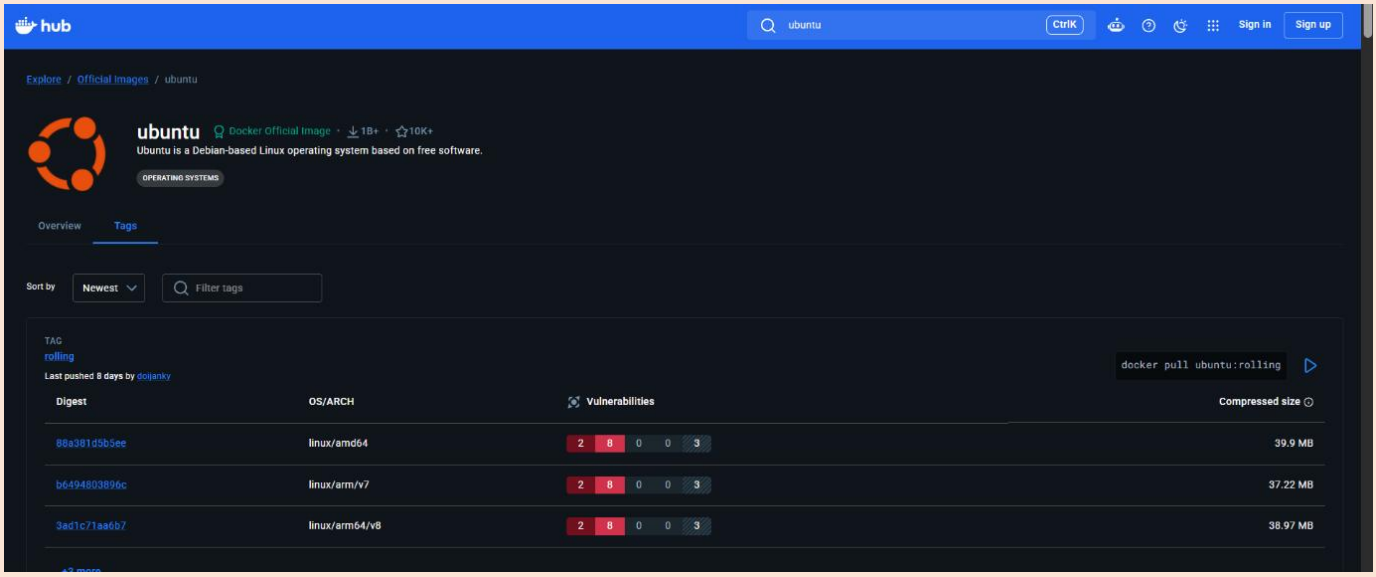
→ docker pull tomcat
→ docker images

Docker-1



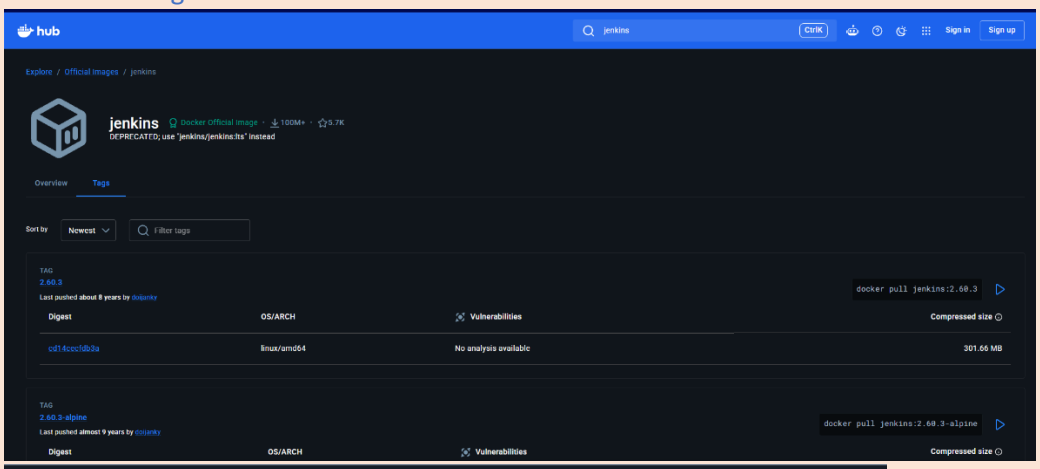


→ docker pull ubuntu
→ docker images



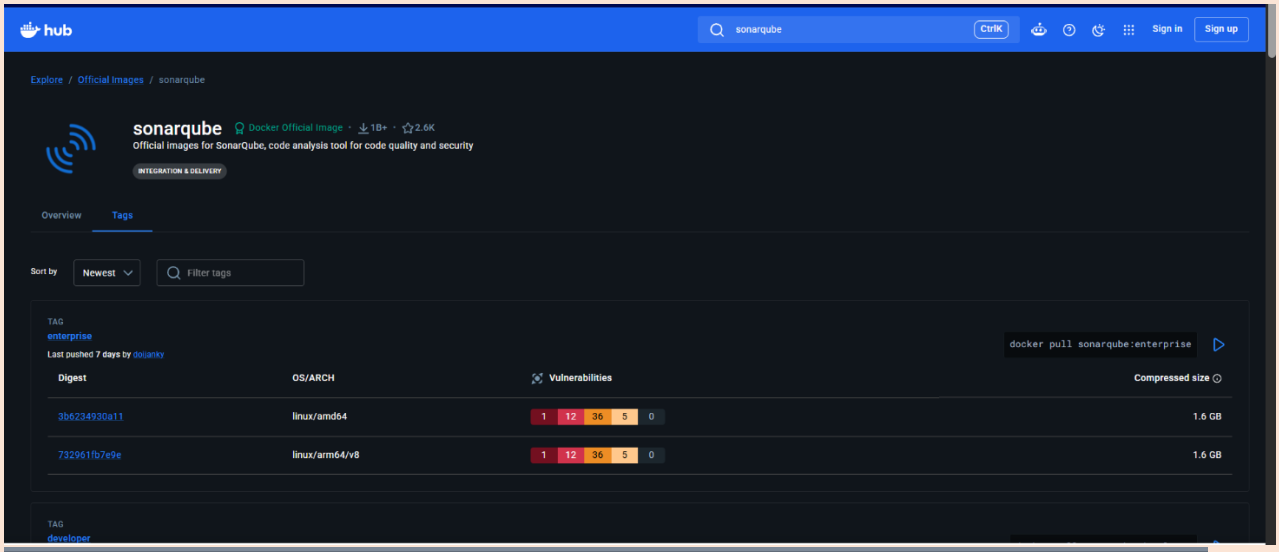
```
[root@ip-172-31-20-30 ec2-user]# docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
4e07a0f12b2c: Already exists
06ad70e463aa: Already exists
Digest: sha256:f144425ff09be612d6d9ad965196e9cdc23dae1f42110a8a11a3e9a8198759f7
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
[root@ip-172-31-20-30 ec2-user]# docker images
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
nginx        latest    57df7715e516   6 days ago    163MB
tomcat       latest    04dd9f6a2a84   6 days ago    444MB
ubuntu       latest    6e92c4fd8dfd   2 weeks ago   101MB
[root@ip-172-31-20-30 ec2-user]#
```

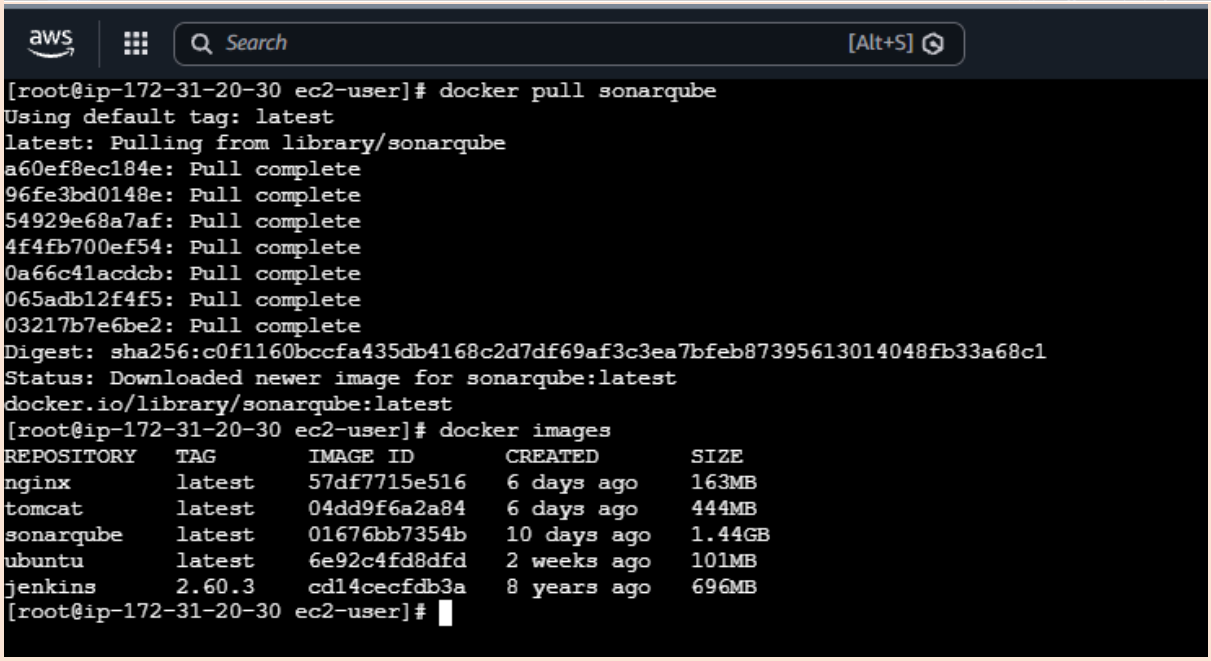
→ docker pull jenkins:2.60.3
→ docker images




```
[root@ip-172-31-20-30 ec2-user]# docker pull jenkins:2.60.3
2.60.3: Pulling from library/jenkins
55cbf04beb70: Pull complete
1607093a898c: Pull complete
9a8ea045c926: Pull complete
d4eee24d4dac: Pull complete
c58988e753d7: Pull complete
794a04897db9: Pull complete
70fcfa476f73: Pull complete
0539c80a02be: Pull complete
54fefc6dcf80: Pull complete
911bc90e47a8: Pull complete
38430d93efed: Pull complete
7e46ccda148a: Pull complete
c0cbcb5ac747: Pull complete
35ade7a86a8e: Pull complete
aa433a6a56b1: Pull complete
841c1dd38d62: Pull complete
b865dcb08714: Pull complete
5a3779030005: Pull complete
12b47c68955c: Pull complete
1322ea3e7bfd: Pull complete
Digest: sha256:eeb4850eb65f2d92500e421b430ed1ec58a7ac909e91f518926e02473904f668
Status: Downloaded newer image for jenkins:2.60.3
docker.io/library/jenkins:2.60.3
[root@ip-172-31-20-30 ec2-user]# docker images
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
nginx        latest    57df7715e516   6 days ago    163MB
tomcat       latest    04dd9f6a2a84   6 days ago    444MB
ubuntu       latest    6e92c4fd8dfd   2 weeks ago   101MB
jenkins      2.60.3    cd14cecfdb3a   8 years ago   696MB
[root@ip-172-31-20-30 ec2-user]#
```

→ docker pull sonarqube
→ docker images



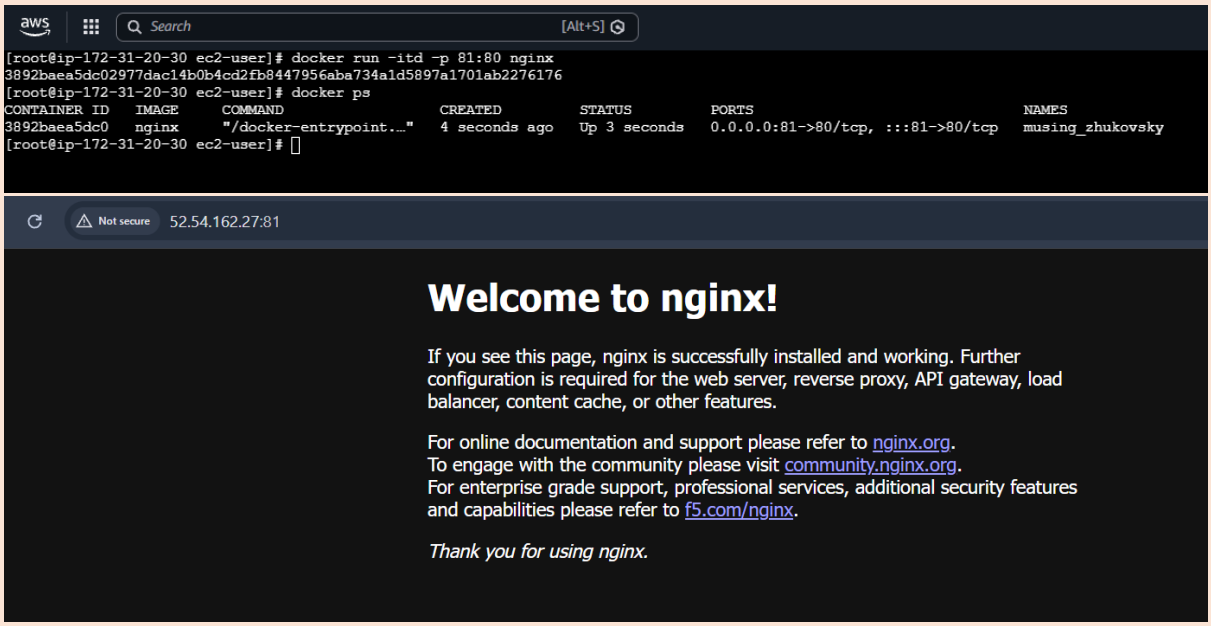


Conclusion: The five Docker images were downloaded to the local Docker environment and made available for inspection and container deployment.

4. Run nginx container and expose on port 81.
   Objective: To run an Nginx container and map host port 81 to container port 80.

   Purpose: To understand Docker port mapping and make the Nginx web server accessible through the EC2 instance.

   → docker run -itd -p 81:80 nginx
   → docker ps



Conclusion: The Nginx container was started with port mapping 81:80. The web server can be accessed at http://<EC2-Public-IP>:81 when the EC2 security group allows inbound traffic on port 81.

5. **Delete the Apache tomcat image from local.**

   Objective: To remove the Apache Tomcat Docker image from the local Docker environment.

   Purpose: To learn Docker image management and understand how to remove unused images to free local storage space

   → docker images
   → docker image rmi IMAGE ID
   →docker images

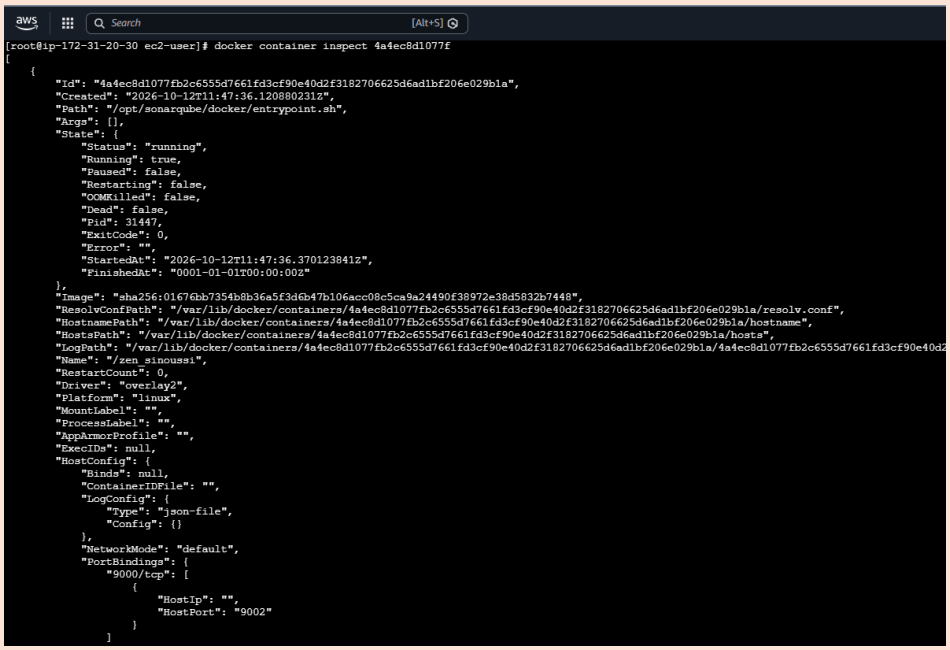
```
[root@ip-172-31-20-30 ec2-user]# docker images
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
nginx        latest    57df7715e516   6 days ago    163MB
tomcat       latest    04dd9f6a2a84   6 days ago    444MB
sonarqube    latest    01676bb7354b   10 days ago   1.44GB
ubuntu       latest    6e92c4fd8dfd   2 weeks ago   101MB
jenkins      2.60.3    cd14cecfdb3a   8 years ago   696MB
[root@ip-172-31-20-30 ec2-user]# docker image rmi 04dd9f6a2a84
Untagged: tomcat:latest
Untagged: tomcat@sha256:bb7c0c78078841047e2d475acce833ef337cb0007ca7c7e4c9ff9058d9019801
Deleted: sha256:04dd9f6a2a84def11e370ff7e96102021043d0c115cb728926202b8621fb94ca
Deleted: sha256:90b02ae599999b3e0a2e9319aefe2d46249cd3a84b04a4256a73fe4ce66a8136
Deleted: sha256:8b4bcc344f173d148c3ca8a755516b8e526f28fcd6a77dca5e28c7873bfa2e76
Deleted: sha256:e434504b45aa7e0af3face17651b8b92ad976aaed02a1e1d0eba614358138aa3
Deleted: sha256:19280388cd8fa31b351a4cf0daa0e9175be81372ed702b33c7aa5eb5bbce8b4e
Deleted: sha256:aed1f647f35084f548ce1265b300b650aa150eaa0bfcf2e99f685c17bc1ab5de
Deleted: sha256:d48beed04406c077ac1cb038fd819d5650881bc4c1aafc17156635551e4503e6
Deleted: sha256:8ee0776a2d1bb912f27e879c3140cc88f29eee8f8b6e093f58205bc190b90443
Deleted: sha256:9aa61dc7dab0b58091261860aefe01de30bcf24fb226890fd1cf936230e7588f
[root@ip-172-31-20-30 ec2-user]# docker images
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
nginx        latest    57df7715e516   6 days ago    163MB
sonarqube    latest    01676bb7354b   10 days ago   1.44GB
ubuntu       latest    6e92c4fd8dfd   2 weeks ago   101MB
jenkins      2.60.3    cd14cecfdb3a   8 years ago   696MB
[root@ip-172-31-20-30 ec2-user]#
```

Conclusion: The Apache Tomcat image was removed from the local Docker environment using Docker image removal commands, provided no existing container was preventing its deletion.

6. Inspect the Jenkins image, SonarQube image.
   Objective: To inspect the SonarQube Docker image and review its configuration details.

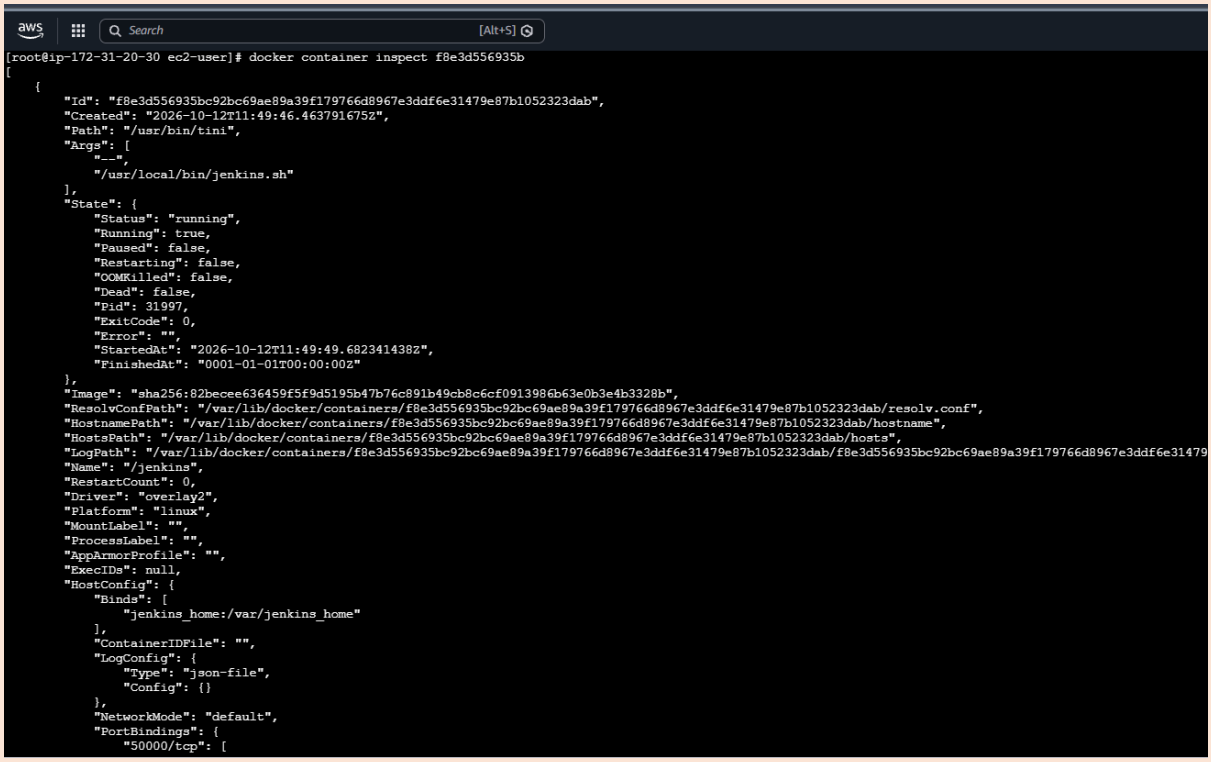
   Purpose: To understand the image metadata, exposed ports, environment variables, and other configuration details required for deploying SonarQube.

   →docker container inspect sonarqube



→docker images
→docker container inspect Jenkins
Objective: To inspect the Jenkins Docker image and examine its configuration and metadata.

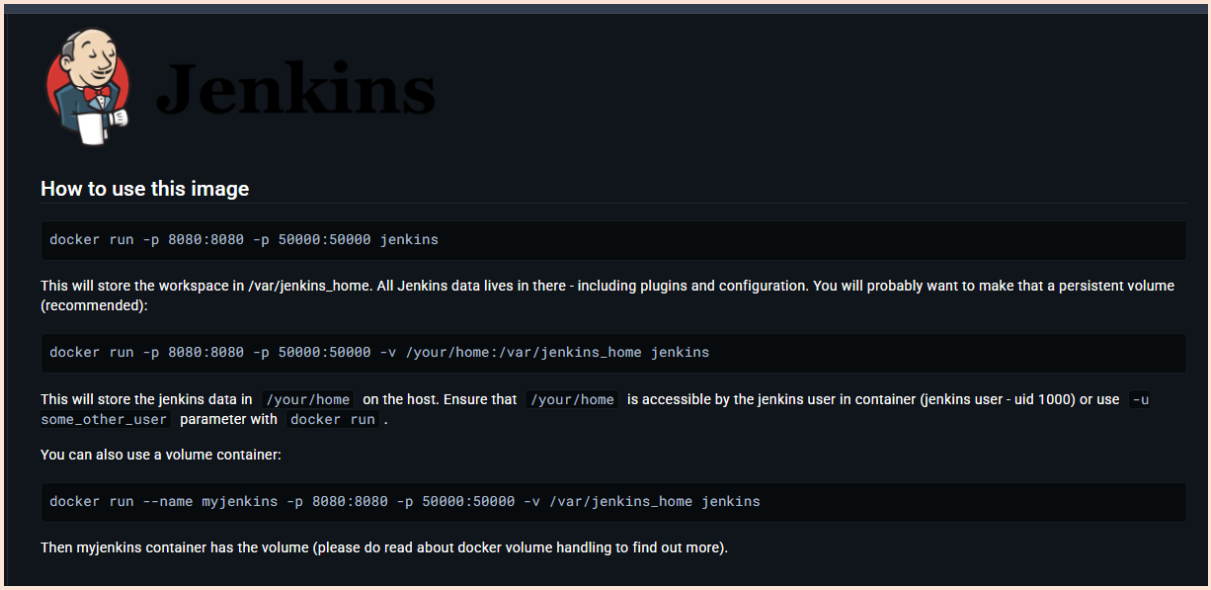
Purpose: To understand image properties such as image ID, tags, layers, environment variables, exposed ports, and startup configuration.



Conclusion: The Jenkins image was inspected using Docker commands, providing an understanding of its metadata and configuration before running a Jenkins container.

7. Run Jenkins container and run one sample job.
Objective: To create and run a Jenkins container using the downloaded Jenkins Docker image.
Purpose: To deploy Jenkins in a containerized environment and learn how to expose its web interface for configuration and pipeline management.

```
docker run -d \
 -p 8081:8080 \
 -p 50000:50000 \
 -v jenkins_home:/var/jenkins_home \
 --restart=always \
 --name jenkins \
 jenkins/jenkins:lts
```



Jenkins

How to use this image

```
docker run -p 8080:8080 -p 50000:50000 jenkins
```

This will store the workspace in /var/jenkins_home. All Jenkins data lives in there - including plugins and configuration. You will probably want to make that a persistent volume (recommended):

```
docker run -p 8080:8080 -p 50000:50000 -v /your/home:/var/jenkins_home jenkins
```

This will store the jenkins data in `/your/home` on the host. Ensure that `/your/home` is accessible by the jenkins user in container (jenkins user - uid 1000) or use `-u some_other_user` parameter with `docker run .`

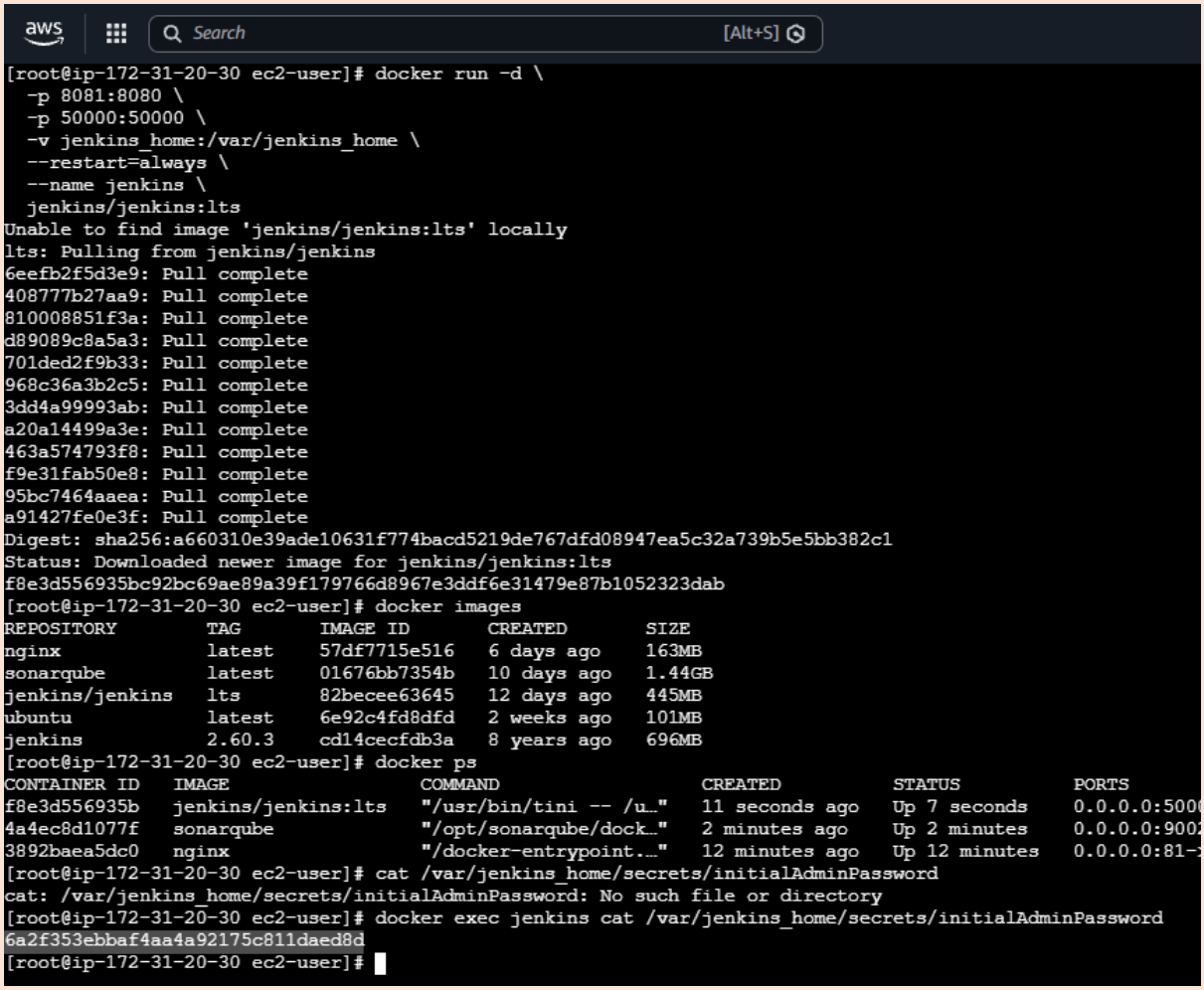
You can also use a volume container:

```
docker run --name myjenkins -p 8080:8080 -p 50000:50000 -v /var/jenkins_home jenkins
```

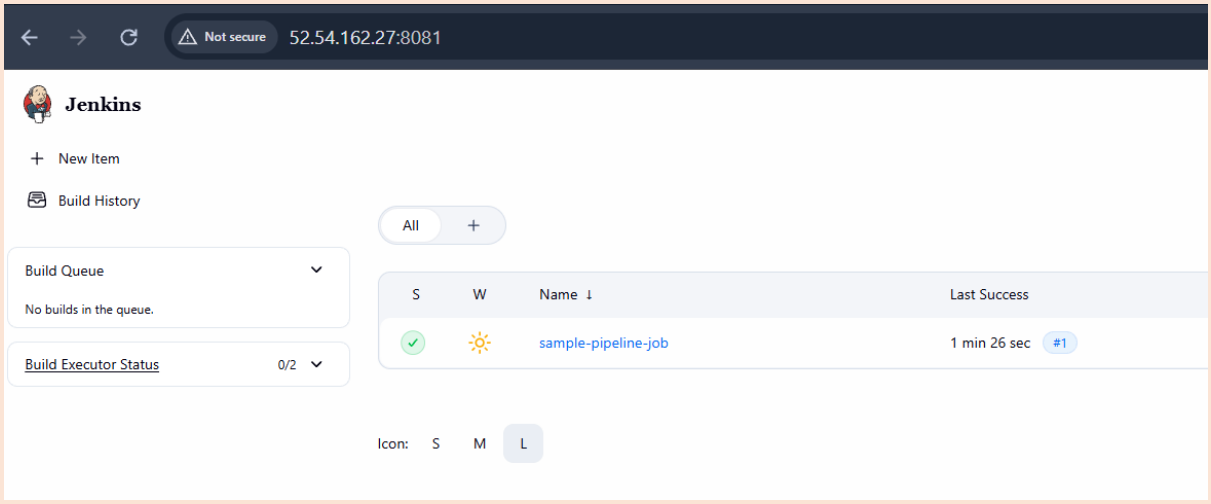
Then myjenkins container has the volume (please do read about docker volume handling to find out more).

→docker ps

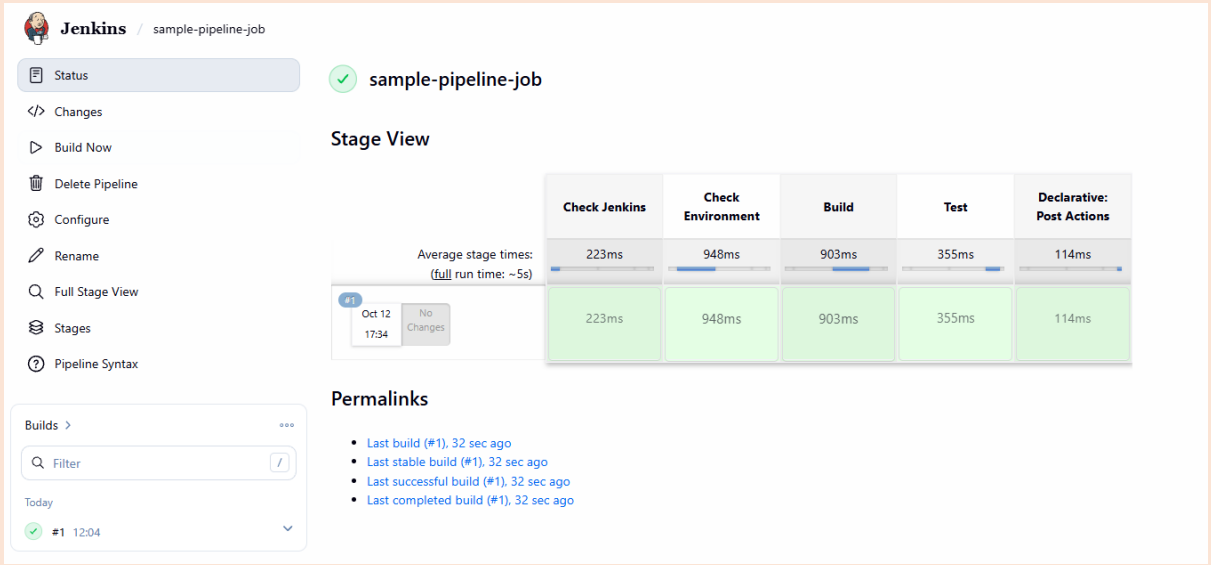
→ to check Ec2 Pub_ip:8081



```
[root@ip-172-31-20-30 ec2-user]# docker run -d \
  -p 8081:8080 \
  -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  --restart=always \
  --name jenkins \
  jenkins/jenkins:lts
Unable to find image 'jenkins/jenkins:lts' locally
lts: Pulling from jenkins/jenkins
6eefb2f5d3e9: Pull complete
408777b27aa9: Pull complete
810008851f3a: Pull complete
d89089c8a5a3: Pull complete
701ded2f9b33: Pull complete
968c36a3b2c5: Pull complete
3dd4a99993ab: Pull complete
a20a14499a3e: Pull complete
463a574793f8: Pull complete
f9e31fab50e8: Pull complete
95bc7464aaea: Pull complete
a91427fe0e3f: Pull complete
Digest: sha256:a660310e39ade10631f774bacd5219de767dfd08947ea5c32a739b5e5bb382c1
Status: Downloaded newer image for jenkins/jenkins:lts
f8e3d556935bc92bc69ae89a39f179766d8967e3ddf6e31479e87b1052323dab
[root@ip-172-31-20-30 ec2-user]# docker images
REPOSITORY        TAG       IMAGE ID       CREATED       SIZE
nginx             latest    57df7715e516   6 days ago    163MB
sonarqube         latest    01676bb7354b   10 days ago   1.44GB
jenkins/jenkins   lts       82becee63645   12 days ago   445MB
ubuntu            latest    6e92c4fd8dfd   2 weeks ago   101MB
jenkins           2.60.3    cd14cecfdb3a   8 years ago   696MB
[root@ip-172-31-20-30 ec2-user]# docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED          STATUS          PORTS
f8e3d556935b   jenkins/jenkins:lts   "/usr/bin/tini -- /u…"   11 seconds ago   Up 7 seconds    0.0.0.0:500
4a4ec8d1077f   sonarqube             "/opt/sonarqube/dock…"   2 minutes ago    Up 2 minutes    0.0.0.0:900
3892baea5dc0   nginx                 "/docker-entrypoint.…"   12 minutes ago   Up 12 minutes   0.0.0.0:81-
[root@ip-172-31-20-30 ec2-user]# cat /var/jenkins_home/secrets/initialAdminPassword
cat: /var/jenkins_home/secrets/initialAdminPassword: No such file or directory
[root@ip-172-31-20-30 ec2-user]# docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
6a2f353ebbaf4aa4a92175c811daed8d
[root@ip-172-31-20-30 ec2-user]#
```



→ created a one sample job



Conclusion: A sample Jenkins Pipeline job was executed to validate the Jenkins setup. A successful build confirmed that Jenkins could execute pipeline stages and report the build result correctly.